

Clean Code与华为编码规范（嵌入式C语言版）

本文档融合了《Clean Code》（Robert C. Martin 著）和华为C语言编码规范的原则，针对嵌入式安全关键系统进行了适配。

1. 函数设计

单一职责

每个函数只做一件事。如果你能用"和"字描述这个函数（例如"验证并发送"），就应该拆分它。

```
/* 反面示例 - 做了两件事 */
int validate_and_send(Packet_t *pkt)
{
    /* 验证逻辑 ... */
    /* 发送逻辑 ... */
}

/* 正面示例 - 每个函数只负责一件事 */
int validate_packet(const Packet_t *pkt)
{
    /* 仅做验证 */
}

int send_packet(const Packet_t *pkt)
{
    /* 仅做发送 */
}
```

抽象层级一致性

同一个函数内的所有操作必须处于相同的抽象层级。不要在同一个函数中混合高层编排逻辑和底层位操作。

```

/* 反面示例 - 混合了不同抽象层级 */
int process_sensor_data(Sensor_t *s)
{
    /* 高层操作 */
    calibrate_sensor(s);

    /* 突然变成底层寄存器访问 */
    uint32_t raw = *(volatile uint32_t *)0x40001234;
    raw = (raw >> 4) & 0xFFFF;

    /* 又回到高层操作 */
    store_result(s, raw);
}

/* 正面示例 - 一致的抽象层级 */
int process_sensor_data(Sensor_t *s)
{
    calibrate_sensor(s);
    uint32_t value = read_sensor_raw(s);
    store_result(s, value);
}

```

参数数量

每个函数最多5个参数。需要更多参数时，将相关参数组合成结构体：

```

/* 反面示例 - 7个参数 */
int uart_init(uint8_t port, uint32_t baud,
              uint8_t data_bits, uint8_t stop_bits,
              uint8_t parity, uint8_t flow_ctrl,
              uint16_t timeout)

/* 正面示例 - 组合成配置结构体 */
typedef struct {
    uint8_t port;
    uint32_t baudrate;
    uint8_t data_bits;
    uint8_t stop_bits;
    uint8_t parity;
    uint8_t flow_ctrl;
    uint16_t timeout_ms;
} UartConfig_t;

int uart_init(const UartConfig_t *cfg)

```

命令-查询分离 (CQS)

函数要么执行操作（命令），要么返回信息（查询），但不能同时做两件事。例外：原子性地修改并返回的操作（如 `queue_pop`）。

```
/* 反面示例 - 既修改状态又返回信息 */
int get_next_item(Queue_t *q, Item_t *out)
{
    q->read_idx++;          /* 副作用! */
    *out = q->buf[q->read_idx];
    return q->count;        /* 返回数量 */
}

/* 正面示例 - 分离 */
bool queue_is_empty(const Queue_t *q);    /* 查询 */
int  queue_pop(Queue_t *q, Item_t *out);  /* 命令 */
```

无副作用

函数不能产生隐藏的副作用。如果一个名为 `validate_xxx` 的函数偷偷修改了全局状态或写入了硬件，那就是一个缺陷。

2. 命名即文档

名称长度与作用域成正比

作用域	命名风格	示例
循环计数器（3行）	短	<code>i, j, n</code>
局部变量	描述性	<code>retry_count</code>
模块级静态变量	非常描述性	<code>s_pending_tx_count</code>
公共函数	完整上下文	<code>UartDriver_SendAsync</code>
宏常量	完整上下文	<code>UART_MAX_BAUD_RATE</code>

命名应揭示意图

名称应当回答：它是什么、为什么存在、如何使用。如果一个名称需要注释来解释，就应该重新命名。

```
/* 反面示例 */
int d;    /* 以天为单位的经过时间 */
int s;    /* 状态标志 */
void p(uint8_t *b, int n);

/* 正面示例 */
int elapsed_days;
int connection_status;
void parse_frame(uint8_t *buf, int buf_len);
```

布尔值命名

布尔值使用 `is_`、`has_`、`can_`、`should_` 前缀：

```
bool is_connected;
bool has_pending_data;
bool can_transmit;
```

避免误导性命名

不要使用暗示不同含义的名称：

```
/* 反面示例 - 实际上不是列表 */
int account_list;

/* 反面示例 - 'hp' 可以是任何含义 */
int hp;

/* 正面示例 */
int account_count;
int health_points;
```

3. 防御性编程（华为规范）

断言用于内部契约

使用断言来检查代码正确运行时必须为真的条件。断言用于捕获 程序员错误，而非运行时错误。

```
#include "platform_assert.h"

void Driver_Send(Driver_t *self,
                 const uint8_t *data,
                 uint16_t len)
{
    ASSERT(self != NULL);
    ASSERT(data != NULL);
    ASSERT(len ≤ TX_BUF_MAX_SIZE);

    /* 确认前置条件满足后继续执行 */
}
```

校验外部输入

外部数据（用户输入、网络数据、传感器读数、来自不可信源的 模块间调用）必须通过运行时检查进行验证，并优雅地处理失败——不能使用断言。

```
int Protocol_ParseFrame(const uint8_t *raw,
                       uint16_t raw_len,
                       Frame_t *out)
{
    if (raw == NULL || out == NULL) {
        return ERR_NULL_PTR;
    }
    if (raw_len < FRAME_MIN_SIZE ||
        raw_len > FRAME_MAX_SIZE) {
        return ERR_INVALID_LENGTH;
    }
    /* 验证通过后进行解析 */
}
```

const正确性

尽可能使用 `const`。这不是可选的——它能防止意外修改并文档化意图。

注意： `const` 用于指针参数表示“不修改指向的数据”，值参数的 `const`（如 `const uint16_t len`）在C语言中没有 实际意义，因为值参数本身是副本，不要对值参数加 `const`。

```
/* 指向const数据的指针（最常见） */
int send(const uint8_t *data, uint16_t len);

/* const指针指向const数据 */
void analyze(const Config_t * const cfg);

/* const结构体成员用于不可变配置 */
typedef struct {
    const uint32_t baudrate;
    const uint8_t port;
} UartFixedConfig_t;
```

static限制作用域

关于 `const` 和 `static` 的完整规则和示例，详见 `04_代码风格.md`（`code-style.md`）对应章节。

4. 错误处理策略（华为规范）

统一错误码体系

定义项目级的错误码枚举。绝不使用裸整数：

```
typedef enum {
    ERR_OK                = 0,
    ERR_NULL_PTR          = -1,
    ERR_INVALID_PARAM     = -2,
    ERR_NO_MEMORY         = -3,
    ERR_TIMEOUT           = -4,
    ERR_BUSY              = -5,
    ERR_HW_FAULT          = -6,
    ERR_BUFFER_FULL       = -7,
    ERR_NOT_INITIALIZED   = -8,
} ErrorCode_t;
```

错误传播

错误向上传播。每一层要么处理错误，要么将其传递给调用者。绝不能静默吞掉错误：

```

/* 反面示例 - 错误被静默忽略 */
void init_system(void)
{
    uart_init(&cfg);      /* 可能失败! */
    sensor_init();        /* 可能失败! */
    start_app();
}

/* 正面示例 - 错误被传播 */
int init_system(void)
{
    int ret;

    ret = uart_init(&cfg);
    if (ret != ERR_OK) {
        return ret;
    }

    ret = sensor_init();
    if (ret != ERR_OK) {
        uart_deinit();    /* 清理已成功初始化的部分 */
        return ret;
    }

    return start_app();
}

```

5. 最小化全局变量

规则

1. 尽可能避免使用全局变量
2. 如不可避免，全局变量加 `g_` 前缀，文件级静态变量加 `s_` 前缀
3. 通过访问函数（getter/setter）保护
4. 文档标明线程安全性要求
5. 绝不使用全局变量进行模块间通信——使用模块的公共API

```

/* 反面示例 - 暴露的全局变量 */
extern volatile uint32_t g_system_tick;

```

```
/* 正面示例 - 访问函数 */
uint32_t System_GetTick(void);
```

6. 表驱动设计（华为规范）

用表查找替代复杂的if-else链和大型switch语句。这能提升可读性、可测试性和可维护性：

```
/* 反面示例 - 冗长的switch */
const char *get_error_string(ErrorCode_t err)
{
    switch (err) {
        case ERR_OK:           return "OK";
        case ERR_NULL_PTR:     return "Null pointer";
        case ERR_TIMEOUT:      return "Timeout";
        /* ... 还有20多个case ... */
        default:               return "Unknown";
    }
}

/* 正面示例 - 表驱动 */
typedef struct {
    ErrorCode_t  code;
    const char  *text;
} ErrorEntry_t;

static const ErrorEntry_t ERROR_TABLE[] = {
    { ERR_OK,           "OK"           },
    { ERR_NULL_PTR,     "Null pointer" },
    { ERR_TIMEOUT,      "Timeout"      },
    /* 轻松添加新条目 */
};

#define ERROR_TABLE_SIZE \
    (sizeof(ERROR_TABLE) / sizeof(ERROR_TABLE[0]))

const char *get_error_string(ErrorCode_t err)
{
    for (uint32_t i = 0; i < ERROR_TABLE_SIZE; i++) {
        if (ERROR_TABLE[i].code == err) {
            return ERROR_TABLE[i].text;
        }
    }
}
```



```
    return "Unknown";  
}
```

适合使用表驱动设计的场景

- 错误码到字符串的映射
- 命令分发（命令ID -> 处理函数指针）
- 状态机转换表
- 配置参数校验范围
- 协议字段解析规则

7. DRY原则（不要重复自己）

如果你发现自己在复制代码，就应该提取成函数。重复的代码意味着重复的Bug和重复的维护工作。

检测方法

注意以下情况： - 带有微小变化的复制粘贴代码块 - 在多处出现的相似switch/if-else结构 - 重复的校验模式 - 重复的资源初始化/清理序列

解决方法

- 将通用逻辑提取为共享函数
- 对变化的行为使用函数指针
- 仅在函数无法胜任时使用宏（例如泛型操作），并优先使用 `static inline` 而非宏

8. 头文件规范

头文件保护、自包含、包含路径和包含顺序的完整规则和示例，详见 `04_代码风格.md`（`code-style.md`）对应章节。

获取更多

- **B站搜索「兆鸣嵌入式」** 观看完整教程（详细讲解+代码实战）
- 公众号搜索「兆鸣嵌入式」 回复「交流」加技术交流群
- 抖音搜索「兆鸣嵌入式」看短视频精华版